

Analisi post-mortem della pipeline di generazione automatica

Confronto tra `finApp` (implementazione manuale) e il progetto generato da GitHub Copilot (`AppFromPrompts`)

NECKER

12 luglio 2026

Sommario

Questo documento analizza l'esito di un esperimento di sviluppo software interamente guidato da agenti AI: la ricostruzione di una web app finanziaria (simulatore di trading con gruppi sociali) a partire dal solo documento `specifiche.md`, tramite una pipeline di prompt sequenziali data in pasto a GitHub Copilot. Il risultato generato (cartella `Desktop/prompts`) viene confrontato con l'implementazione manuale di riferimento (`finApp`). L'analisi quantifica cosa ha funzionato (type-safety perfetta, copertura funzionale ampia, disciplina di processo), cosa è fallito (81 suite di test su 265 non passano, tripla implementazione della stessa logica di business, architettura a code costruita ma mai collegata, integrazione posticcia a fine progetto) e ricostruisce le cause radice, con particolare attenzione ai meccanismi di *context rot* — la degradazione progressiva della coerenza quando più agenti lavorano in isolamento su contesti parziali. Il documento si chiude con raccomandazioni operative per la prossima iterazione della pipeline.

Indice

1	Introduzione e obiettivo	3
2	La pipeline utilizzata	3
2.1	Fase 1 — Espansione delle specifiche in quattro documenti tecnici	3
2.2	Fase 2 — Decomposizione di ogni documento in feature atomiche	3
2.3	Fase 3 — Controllo di coerenza dei documenti feature	4
2.4	Fase 4 — Creazione del <code>MASTER_SYSTEM_CONTEXT</code>	4
2.5	Fase 5 — Implementazione di ogni feature (un agente per feature)	4
2.6	Fase 6 — QA per feature (Gap Analysis)	4
2.7	Fase 7 — Test di integrazione per documento	4
3	Metodologia di analisi	4
4	Stato del progetto generato: le metriche	5
4.1	Quadro d'insieme	5
4.2	Compilazione: verde	5
4.3	Test: rosso, nonostante i resoconti dicano il contrario	5
4.4	Timeline dei commit	5
5	Cosa ha funzionato	6
6	Cosa è andato storto	6

6.1	La tripla implementazione della logica di business	6
6.2	L'architettura a code: costruita, testata, mai collegata	7
6.3	Il monolite API e l'app che girava sui mock	7
6.4	Il breakdown dei test falliti	8
6.5	RBAC: il middleware c'è, le rotte non lo usano	8
6.6	Amplificazione documentale e feature inventate	8
6.7	Struttura di progetto anomala	9
6.8	QA a copertura diseguale	9
7	Confronto con finApp	9
7.1	Filosofie opposte	9
7.2	Osservazioni chiave	9
8	Analisi delle cause radice: dove ha agito il context rot	10
8.1	Causa 1 — L'isolamento degli agenti senza contratti condivisi eseguibili	10
8.2	Causa 2 — Integrazione come fase finale anziché vincolo continuo	10
8.3	Causa 3 — “Verde” locale scambiato per “verde” globale	10
8.4	Causa 4 — L'espansione documentale come moltiplicatore di allucinazioni	10
8.5	Causa 5 — Regole globali che perdono contro le istruzioni locali	11
9	Raccomandazioni per la prossima iterazione della pipeline	11
10	Conclusioni	12
A	Appendice A — Esito completo della test suite	13
B	Appendice B — Inventario delle duplicazioni	13
C	Appendice C — Cronologia essenziale del progetto generato	13

1 Introduzione e obiettivo

L'esperimento nasce da una domanda precisa: *è possibile ricreare una web app reale, già costruita a mano, usando esclusivamente una pipeline di prompt strutturati e agenti di coding, senza incorrere nei classici errori da context rot?*

I due artefatti confrontati sono:

- **finApp** — l'implementazione manuale di riferimento: backend Express + Prisma + PostgreSQL, frontend Vite + React (SPA con React Router), immagini su Cloudinary, motore di trading a `setInterval`. Circa 16.800 righe di TypeScript, 30 commit tra il 19 marzo e il 4 aprile 2026.
- **Desktop/prompts** — il progetto generato dalla pipeline: monorepo con backend Express + Prisma e frontend Next.js (App Router) annidato in `src/frontend`, circa 23.500 righe di sorgente più 26.400 righe di test (265 file di test), 384 commit tra il 22 febbraio e il 19 marzo 2026.

Il verdetto sintetico, anticipato: **la pipeline ha prodotto un'enorme quantità di codice di buona qualità locale, ma non un'applicazione integrata e funzionante end-to-end**. Ogni feature, presa da sola, è ben scritta, tipata e testata; il sistema nel suo complesso contiene però tre implementazioni parallele della stessa logica di business, un'infrastruttura a code interamente morta, e una fase di integrazione finale improvvisata che ha rotto test precedentemente verdi senza che nessun agente se ne accorgesse. Il context rot non si è manifestato come "l'agente dimentica le istruzioni", ma come **divergenza sistemica tra mondi paralleli costruiti da agenti che non si sono mai parlati**.

2 La pipeline utilizzata

La pipeline era composta da sette tipologie di prompt, applicate in sequenza. Se ne dà qui una descrizione dettagliata, perché la struttura stessa della pipeline è parte delle cause analizzate più avanti.

2.1 Fase 1 — Espansione delle specifiche in quattro documenti tecnici

Un prompt con ruolo "Software Architect e Tech Lead Senior" riceve `specifiche.md` (327 righe) e genera quattro documenti tecnici "progressivi": (1) architettura e costruzione del database, (2) motore backend e algoritmi core, (3) infrastruttura API e sicurezza, (4) frontend e UI/UX. La regola fondamentale imposta era: *"NON SINTETIZZARE. Ogni documento deve essere lungo, estremamente dettagliato e includere ogni singola regola di business"*.

Risultato: 4 documenti per un totale di **11.154 righe** (rispettivamente 1.337, 3.585, 2.329 e 3.903 righe) — un fattore di espansione di circa 34× rispetto alla specifica originale.

2.2 Fase 2 — Decomposizione di ogni documento in feature atomiche

Ogni documento viene scomposto in "feature atomiche indipendenti", ognuna con un proprio documento di specifica pensato come system prompt per un agente TDD. Requisiti chiave: ciclo Red-Green-Refactor stretto, commit Conventional Commits a ogni ciclo verde, e soprattutto *"ogni documento deve permettere a un agente di lavorare senza conoscere le altre feature"*.

Risultato: **42 feature totali** — 12 per il documento database, 12 per il backend, 10 per le API, 8 per il frontend.

2.3 Fase 3 — Controllo di coerenza dei documenti feature

Un prompt di verifica chiedeva se, svolgendo tutte le feature con un agente ciascuna, si sarebbe ottenuta l'app completa in modo ordinato e coerente. Per il documento 4 questa fase ha effettivamente individuato problemi reali (componente `GroupCard` triplicato tra tre feature, dipendenze non dichiarate tra feature 3, 5 e 8) e ha prodotto due artefatti correttivi: `FIX_REPORT.md` e `SHARED_CONTRACTS.md`, quest'ultimo con i contratti TypeScript condivisi. È significativo che questi artefatti esistano *solo* per il documento 4: per i documenti 1–3 la verifica non ha lasciato tracce equivalenti.

2.4 Fase 4 — Creazione del MASTER_SYSTEM_CONTEXT

Un prompt di sintesi ha condensato specifiche + 4 documenti in un file di contesto per gli agenti (`MASTER_SYSTEM_CONTEXT.md`, 329 righe): regole di business non negoziabili, matrice ruoli/permessi, schema DB essenziale, riferimento endpoint, flusso del motore di trading, stack tecnologico (con divieto esplicito del Pages Router di Next.js), codici d'errore con i testi esatti, linee guida DO/NEVER — inclusa la regola in maiuscolo sul **mocking obbligatorio** di code, Redis e API esterne nei test.

2.5 Fase 5 — Implementazione di ogni feature (un agente per feature)

Ogni agente riceveva come “intero universo di conoscenza” esclusivamente il proprio file feature e il master context, con istruzione esplicita di ignorare ogni altra convenzione. Doveva committare all'inizio (per separare le feature), poi a ogni verde, autoverificarsi a fine lavoro e produrre un file di resoconto (`RESOCONTO_FEATURE_*.md`).

2.6 Fase 6 — QA per feature (Gap Analysis)

Un prompt con ruolo “Senior QA Engineer” confrontava l'implementazione con il documento PRD, producendo un report diagnostico per gravità (BLOCCANTE / MAGGIORE / MINORE) su gap, vincoli di dominio, RBAC e edge case.

2.7 Fase 7 — Test di integrazione per documento

Completate tutte le feature di un documento, un prompt “SDET” generava una suite di integrazione (Jest + Supertest + Prisma) su isolamento dei flussi, errori di dominio, RBAC e mocking dei servizi esterni. Ogni fase era seguita da eventuali cicli di correzione se i controlli fallivano.

3 Metodologia di analisi

L'analisi è stata condotta direttamente sui due repository, con misure riproducibili:

- **Compilazione:** `npx tsc -noEmit` sull'intero progetto generato.
- **Test:** esecuzione completa di `npx jest` (265 suite), con classificazione dei fallimenti per causa (errore d'ambiente vs regressione reale) tramite ispezione dei log.
- **Storia git:** distribuzione temporale dei 384 commit, aderenza a Conventional Commits, pattern “commit a ogni verde”.
- **Analisi strutturale:** mappatura di servizi, repository, worker, middleware e relative dipendenze di import, per individuare codice morto e duplicazioni.
- **Confronto documentale:** specifiche originali vs documenti generati vs codice, e artefatti di processo (`TODO.md`, `VERIFICA_SPECIFICHE.md`, report diagnostici, resoconti feature).

- **Confronto con finApp:** schema Prisma, architettura runtime, struttura del frontend, gestione cron/immagini.

4 Stato del progetto generato: le metriche

4.1 Quadro d'insieme

Metrica	prompts (generato)	finApp (manuale)
Righe TypeScript sorgente	~23.556	~16.817
Righe di test	~26.391 (265 file)	0
Commit	384 (22 feb – 19 mar)	30 (19 mar – 4 apr)
Endpoint REST	68 (in un unico file)	~60 (per dominio, 5 router)
Migrazioni Prisma	12	13
Modelli Prisma	11 + 5 enum	12 + 5 enum
Trigger DB	subset (in 6 migrazioni)	migrazione dedicata
Frontend	Next.js App Router in <code>src/frontend</code>	Vite + React Router, repo separato
Coda ordini	BullMQ/Redis (mai collegata)	<code>setInterval</code> 60s

Tabella 1: Confronto quantitativo tra i due progetti.

4.2 Compilazione: verde

`tsc -noEmit` termina con **0 errori** su circa 50.000 righe complessive. È un risultato non banale e va riconosciuto: il vincolo TypeScript strict imposto dal master context ha retto attraverso 42 agenti diversi.

4.3 Test: rosso, nonostante i resoconti dicano il contrario

L'esecuzione completa della suite dà:

81 suite fallite / 183 passate / 1 saltata — **370 test falliti** / 589 passati su 963

Ogni singolo `RESOCONTO_FEATURE_*.md` dichiara invece la propria suite verde, e il report diagnostico del documento 1 (datato 17 marzo) certifica “75 passed, 1 skipped”. Questo scarto tra *stato dichiarato* e *stato reale* è il singolo dato più importante dell'intera analisi, e viene scomposto nella sezione 6.4.

4.4 Timeline dei commit

La distribuzione temporale racconta come è andato davvero il lavoro:

Giorno	Commit	Attività prevalente
22–23 feb 2026	14	Feature frontend (doc 4): profilo, immagini, primo scheletro Next.js
13 mar 2026	18	Ripresa: guard DB amicizie (doc 1)
16 mar 2026	23	Feature doc 1 (stock, portfolio, transazioni)
17 mar 2026	200	Grosso delle feature doc 1–3 + gap closure
18 mar 2026	128	Feature restanti, wiring, remediation TODO
19 mar 2026	1	Ultimo ritocco

Tabella 2: Commit per giorno nel repository generato.

Metà del progetto (328 commit su 384) è stata generata in 48 ore. In quelle stesse 48 ore sono state prese tutte le decisioni di integrazione — ed è lì che si concentrano i danni.

5 Cosa ha funzionato

È importante essere onesti sui successi, perché definiscono cosa *conservare* della pipeline.

1. **Type-safety totale.** Zero errori di compilazione su 50k righe scritte da 42 agenti diversi. Il vincolo “TypeScript + Prisma per tutto” nel master context ha funzionato da contratto implicito più efficace di qualunque prosa.
2. **Fedeltà dello schema dati.** Lo schema Prisma generato è sostanzialmente sovrapponibile a quello di `finApp`: stesse entità (Persona, Gruppo, Portafoglio, Transazione, Azioni in possesso, Amicizia, Membro/Invito gruppo, Watchlist, Storico), stessi enum, uso corretto e sistematico di `Decimal` per i valori monetari (mai `Float`). La regola “DECIMAL, mai float” — ripetuta in specifiche, master context e singole feature — è stata rispettata ovunque. **La ridondanza mirata sulle regole critiche paga.**
3. **Copertura funzionale del frontend.** Il frontend Next.js copre tutte le pagine della specifica: homepage pubblica, login/registrazione, area personale, pagina azione con pannello di trading, area gruppi con dettaglio, pagina social, overview persona, impostazioni account. Le rotte dinamiche (`groups/[id]`, `stock/[id]`, `user/[id]`) sono corrette, il divieto del Pages Router è stato rispettato, Zustand e React Query sono usati come da specifica, lo smart polling degli ordini pendenti esiste (`useSmartPolling.ts`).
4. **Disciplina di processo.** I 384 commit sono realmente Conventional Commits, realmente granulari (un ciclo verde $\approx$ un commit), con baseline `chore`: a inizio feature. La storia git è leggibile e permette di ricostruire l’ordine di costruzione — è proprio grazie a questa disciplina che l’analisi è stata possibile.
5. **Trigger e vincoli a livello DB (parziale).** Le migrazioni contengono trigger reali: auto-ordinamento e unicità delle amicizie, no self-friendship, cancellazione automatica dell’invito all’ingresso nel gruppo, single-owner, check constraint su coerenza Buy/Sell. Il ciclo QA del documento 1 ha aggiunto in “gap closure” il trigger sui permessi di trading e i check anti-bypass con `IS NOT NULL`.
6. **Il ciclo QA ha trovato problemi veri.** Il report diagnostico del doc 1 ha individuato correttamente: trigger mancanti, rischio IDOR sulla creazione ordini (nessuna verifica di ownership del portafoglio), divergenza RBAC Admin/Owner sulla modifica saldi, edge case sul cancel di ordini sell. Le prime correzioni sono state applicate e testate. Anche la fase 3 (coerenza feature doc 4) ha intercettato la triplicazione di `GroupCard` *prima* dell’implementazione, producendo `SHARED_CONTRACTS.md`.
7. **Il TODO finale come radiografia.** `TODO.md` — generato nella fase di allineamento finale — è un documento di qualità: elenca con precisione mock da sostituire, endpoint sbagliati, funzioni client mancanti, violazioni architetturali (grafici che dovevano essere delegati a TradingView), bug di logica (perdita del prezzo medio all’annullamento di una vendita). Quasi tutto risulta poi effettivamente corretto e spuntato.

6 Cosa è andato storto

6.1 La tripla implementazione della logica di business

È il fallimento più grave e il più istruttivo. Nel progetto convivono **tre implementazioni parallele degli stessi concetti** (Tabella 3):

I due servizi “Portfolio” non condividono nulla: uno usa `Prisma.Decimal` e i codici errore di `types/errors.ts`, l’altro usa `decimal.js` e una gerarchia di errori propria (`PortfolioServiceErrors`), con concetti inventati come `DEFAULT_GROUP_BUDGET_LIMIT`. Nessuno dei due è sbagliato in sé — sono

Origine	Convenzione	Artefatti
Feature doc 1 (database)	Italiano, <code>nome.service.ts</code>	<code>portfolio.service.ts</code> , <code>transazione.service.ts</code> , 11 repository in <code>src/repositories/</code> (<code>persona</code> , <code>portafoglio</code> , ...)
Feature doc 2 (backend)	Inglese, <code>PascalCase.ts</code>	<code>PortfolioService.ts</code> , <code>TradingService.ts</code> , 4 repository in <code>src/database/repositories/</code>
Wiring finale (fase TODO)	Inline, prisma diretto	<code>runtimeDependencies.ts</code> : 1.317 righe che reimplementano ordini, saldi, snapshot con query Prisma dirette

Tabella 3: Le tre implementazioni coesistenti della stessa logica.

mondi coerenti ma incompatibili, costruiti da agenti a cui era stato ordinato di ignorare tutto ciò che non fosse il proprio file feature. Il problema è riconosciuto anche internamente: l'ultimo item aperto del `TODO.md` recita “*C’è logica ridondante o non unificata tra `TransazioneService.executeOrder` e `TransactionWorker.ts`”.*

6.2 L’architettura a code: costruita, testata, mai collegata

Le feature 1–3 del documento 2 hanno costruito l'intero impianto BullMQ/Redis: `src/queue/` (connessione, code, scheduler, tipi), `TransactionWorker.ts` (366 righe), `SnapshotWorker.ts`, con test unitari verdi e mock conformi alla regola del master context. Poi, al momento del cablaggio reale, `runtimeDependencies.ts` ha implementato il motore di trading con due semplici `setInterval` (sweep ordini e snapshot), chiamando direttamente `TransazioneService.executeOrder`. Risultato: **l'intera infrastruttura a code è codice morto** — nessun file di runtime importa i worker o le code.

L'ironia è che la scelta del `setInterval` è quella *giusta*: è esattamente ciò che fa `finApp` (cron interno a 60 secondi in `tradingEngine.ts`). Il problema non è la soluzione finale, ma che la pipeline abbia fatto costruire e testare per giorni un'infrastruttura che l'integrazione ha poi ignorato, senza che nessuno la rimuovesse o la collegasse.

6.3 Il monolite API e l'app che girava sui mock

`createApiApp.ts` è un file di **2.553 righe** contenente tutti i 68 endpoint. Definisce un proprio contratto di dipendenze (`ApiDependencies`: `authService`, `portfolioService`, `tradingService`, `marketService`, `adminService`) che **non corrisponde a nessuno dei servizi costruiti dalle feature dei documenti 1 e 2**: è una terza tassonomia, inventata dall'agente della feature “API routes”. Per farla girare, `src/index.ts` forniva `createDevDependencies()`: dati finti hardcoded (portafoglio con AAPL e MSFT, token `dev-access-token`, ricerche che ritornano array vuoti).

In pratica, **per la maggior parte della vita del progetto l'app “funzionava” su dati finti**, e tutti i test HTTP passavano contro i mock. Solo nella fase finale (`TODO.md`, item 1) i mock sono stati sostituiti dal bootstrap reale con Prisma — ed è in questa saldatura tardiva che si è concentrato il caos: la fase di wiring ha dovuto scoprire e correggere in un colpo solo decine di discrepanze (path errati, metodi client mancanti, cron non collegato, snapshot senza asset congelati).

6.4 Il breakdown dei test falliti

Le 81 suite fallite si dividono in due famiglie molto diverse:

Famiglia	Suite	Causa
Test feature doc 1 (friendship, watchlist, user, transaction, stock, portfolio, permission, membership, invitation, holdings, historical, group: 6-7 suite ciascuno)	~73	PrismaClientInitializationError: i test richiedono un PostgreSQL reale con credenziali locali (868 errori “Authentication failed against database server”). Non riproducibili fuori dalla macchina/momento in cui furono scritti.
Test frontend e integrazione (homepage it. 1, personal-dashboard it. 1/8/11, social-interface it. 5, doc4-remediation)	~8	Regressioni reali: i commit successivi di wiring e remediation (es. 6b83485 “close feature3 wiring gaps”) hanno modificato componenti — rimosso <code>testid</code> come <code>portfolio-overview-skeleton</code> , cambiato composizioni — senza rieseguire le suite delle iterazioni precedenti.

Tabella 4: Classificazione delle 81 suite fallite.

Entrambe le famiglie sono fallimenti *di pipeline*, non di sfortuna:

- La prima è una **violazione diretta della regola più enfatizzata del master context** (“**MOCKING OBBLIGATORIO** [...] i test non devono MAI tentare connessioni di rete reali”): gli agenti del documento 1 hanno scritto sistematicamente test d’integrazione contro un Postgres vivo. La regola scritta in maiuscolo non è bastata, perché i loro file feature (il loro “universo di conoscenza”) descrivevano trigger e vincoli DB che *hanno senso solo* su un database reale: quando istruzione locale e regola globale configgono, l’agente segue l’istruzione locale.
- La seconda è l’assenza di un **regression gate**: nessun agente delle fasi successive aveva l’obbligo di rieseguire l’intera suite prima di committare. Ognuno verificava solo i propri test (“verdi”), e le rotture altrui restavano invisibili. È la formalizzazione perfetta del context rot: *ciò che è fuori dal contesto dell’agente non esiste, nemmeno quando è un test rosso*.

6.5 RBAC: il middleware c’è, le rotte non lo usano

La feature 2 del documento 3 ha costruito `rbacGroupMiddleware.ts` + `PermissionSystem` con test verdi. Le rotte di `createApiApp.ts` però eseguono controlli di ruolo manuali (`getEffectiveGroupRole`) — item ancora aperto nel TODO. Stesso pattern della coda: **il componente giusto esiste, l’integrazione lo ignora**. La falla “Admin espelle Admin” è stata poi chiusa, ma nel service layer, creando un ulteriore punto di verità sui permessi (middleware, service, controlli inline).

6.6 Amplificazione documentale e feature inventate

Il divieto di sintesi in fase 1 ha trasformato 327 righe di specifiche in 11.154 righe di documenti. In quell’espansione $34\times$, ogni documento ha *inventato* requisiti plausibili ma mai richiesti, che le fasi successive hanno trattato come obblighi contrattuali, producendo feature intere:

- doc 2, feature 11: *Performance Monitoring* (`PerformanceCollector`, `PerformanceAnalytics`, `AlertManager`, `HealthCheckService`);
- doc 3, feature 9–10: *Error Handling & Resilience* (circuit breaker, retry con backoff) e *Rate Limiting & Security* (`ddosProtector`, `securityAuditLogger`, rate limiter adattivo);

- doc 4, feature 7: achievements utente, privacy settings, grafici performance con timeframe — assenti dalle specifiche;
- il master context ha aggiunto di suo la strategia dual-API “Finnhub real-time + Polygon.io daily” (le specifiche citavano Finnhub o Polygon come alternative) e un `AnalyticsWebSocketService` è comparso nel codice nonostante la specifica prescrivesse esplicitamente short polling.

Una quota rilevante delle ~50.000 righe generate — plausibilmente un quarto — serve requisiti che non esistono. Ogni riga inventata è costata tempo di generazione, test, QA, e ha allargato la superficie su cui il context rot poteva agire.

6.7 Struttura di progetto anomala

Un unico `package.json` mescola Express, Prisma, BullMQ, Next.js, React 19 e le testing library; il frontend vive in `src/frontend` dentro il source tree del backend (con la cartella `.next` committata insieme ai sorgenti); i test in tre posti diversi (`src/tests/`, `*.test.ts` accanto ai file, `__tests__/`). È l’accumulo di 42 decisioni locali mai riconciliate — nessun agente aveva l’autorità (o il contesto) per dire “il frontend è un progetto separato”. `finApp`, per confronto, ha due package indipendenti con confini netti.

6.8 QA a copertura diseguale

Il ciclo diagnostico previsto dalla fase 6 risulta documentato solo per il documento 1: in `reports/` esiste unicamente `REPORT_DIAGNOSTICO_DOC1_GAP_ANALYSIS.md` (con due cicli di gap closure). Per i documenti 2, 3 e 4 non esistono report equivalenti nel repository. Proprio le aree senza QA formale (API wiring, frontend) sono quelle in cui si concentrano le regressioni reali.

7 Confronto con finApp

7.1 Filosofie opposte

	finApp (manuale)	prompts (generato)
Filosofia	Minimo indispensabile che funziona	Massimo teorico descritto nei documenti
Motore di trading	<code>setInterval</code> 60s + endpoint protetti da <code>CRON_SECRET</code>	BullMQ/Redis (morto) + <code>setInterval</code> (attivo)
Frontend	Vite SPA, React Router, repo separato	Next.js App Router dentro il backend
Immagini	Cloudinary con crop client	<code>OptimizedImage</code> + placeholder SVG generativi (nessun provider reale collegato)
Test	0 (verifica manuale + <code>API_ROUTES_CHECKLIST.md</code>)	963 (38% falliti)
Sicurezza	JWT + middleware auth/cron, helmet, rate limit	JWT + RBAC + audit + DDoS protector (integrazione parziale)
Fonte di verità	Il codice stesso + checklist	5 documenti + 42 feature + master context

Tabella 5: Confronto architetturale e di metodo.

7.2 Osservazioni chiave

1. La spec era più ambiziosa dell’app reale — e la pipeline non poteva saperlo. `specifiche.md` descrive Redis, BullMQ, caching multilivello, tre formati d’immagine WebP. `finApp` non implementa quasi nulla di tutto ciò: niente Redis, niente code, cron semplice. Chi ha

scritto la spec sapeva quali parti fossero aspirazionali; la pipeline, per costruzione (“non omettere nulla”), le ha trattate tutte come requisiti vincolanti e le ha pure amplificate. **Il documento di specifica dato a una pipeline AI deve distinguere il “must” dal “nice to have”,** perché l’AI non ha il contesto per farlo da sola.

2. Dove la spec era precisa, il generato è allineato. Schema dati, messaggi d’errore testuali, flusso ordini pending/executed, isolamento privato/gruppo: su tutto ciò che le specifiche definivano in dettaglio, i due progetti convergono in modo notevole. La qualità dell’output è direttamente proporzionale alla precisione dell’input.

3. Il generato è “più completo” sulla carta, meno funzionante in pratica. Il progetto generato ha più endpoint, più vincoli DB dichiarati, test, monitoring, resilienza. Ma `finApp` è un’applicazione che gira coerentemente end-to-end, mentre il generato è un insieme di ottimi moduli con saldature fragili. Per un simulatore di trading, l’integrità end-to-end del flusso ordine→esecuzione→saldo vale più di qualunque circuit breaker.

8 Analisi delle cause radice: dove ha agito il context rot

I sintomi descritti sopra discendono da cinque cause strutturali della pipeline.

8.1 Causa 1 — L’isolamento degli agenti senza contratti condivisi eseguibili

La regola “ogni agente lavora solo col suo file feature + master context” è stata pensata proprio per combattere il context rot: contesti piccoli, freschi, focalizzati. Ha però eliminato l’unico blocco alla divergenza: la *conoscenza reciproca*. Il master context descriveva regole di business e stack, ma non i **contratti concreti**: nomi dei file, firme dei servizi, convenzione di naming, struttura delle cartelle. Ogni agente ha quindi inventato la propria tassonomia, e 42 tassonomie non compongono un sistema. La prova a contrario esiste nel progetto stesso: dove un contratto condiviso c’era (`SHARED_CONTRACTS.md` per il doc 4; lo schema Prisma per tutti), la convergenza c’è stata.

8.2 Causa 2 — Integrazione come fase finale anziché vincolo continuo

La pipeline costruiva prima tutti i pezzi e poi li saldava (mock in `index.ts` → wiring finale). Ogni errore di interfaccia è rimasto invisibile finché tutti i pezzi non sono stati appoggiati uno sull’altro, e a quel punto la correzione è avvenuta sotto pressione, in un contesto ormai enorme, da parte di agenti che non conoscevano le feature originali: è il momento esatto in cui sono nate la terza implementazione inline (6.1) e le regressioni frontend. In termini di context rot: **più a lungo si rinvia l’integrazione, più contesto serve per farla, e meno contesto è disponibile.**

8.3 Causa 3 — “Verde” locale scambiato per “verde” globale

Il protocollo TDD imponeva di committare a ogni verde *della propria feature*. Nessuna fase imponeva di rieseguire tutto. I resoconti sono quindi tutti sinceramente verdi e il progetto è complessivamente rosso. Senza un gate automatico esterno agli agenti (CI), l’autocertificazione non ha valore probatorio: l’agente riporta ciò che vede nel suo contesto, e nel suo contesto era tutto verde davvero.

8.4 Causa 4 — L’espansione documentale come moltiplicatore di allucinazioni

Ogni passaggio generativo documento→documento aggiunge dettagli plausibili non presenti alla sorgente; il passaggio successivo li tratta come requisiti. Con quattro salti (specifiche →

documenti → feature → master context → codice), le invenzioni si sono accumulate e *legittimate a vicenda* (il rate limiting inventato dal doc 3 è rientrato nel master context, che l’ha reso obbligatorio per tutti). È context rot in forma documentale: la fonte di verità si allontana a ogni copia.

8.5 Causa 5 — Regole globali che perdono contro le istruzioni locali

La regola del mocking obbligatorio, pur scritta in maiuscolo nel master context, è stata ignorata da 12 feature su 12 del documento 1 — perché i loro file feature richiedevano di verificare trigger e vincoli che esistono solo nel database reale. Quando la regola globale contraddice il compito locale, l’agente risolve il conflitto a favore del compito. Le regole trasversali vanno quindi rese *impossibili da violare* (ambienti, fixture, script di test forniti) invece che *proibite a parole*.

9 Raccomandazioni per la prossima iterazione della pipeline

1. **Sostituire l’espansione con la strutturazione.** Non chiedere quattro documenti “lunghi ed esaustivi che non omettono nulla”: chiedere documenti che *riorganizzino* le specifiche senza aggiungere requisiti, con regola inversa (“se un requisito non è nelle specifiche, non può comparire; se ritieni manchi qualcosa, elencalo in una sezione OPEN QUESTIONS”). Marcare nelle specifiche cosa è vincolante e cosa è aspirazionale.
2. **Contratti prima delle feature (contract-first).** Prima di lanciare qualunque agente implementatore, generare e congelare: schema Prisma, elenco endpoint con DTO request/response (es. OpenAPI), **interfacce TypeScript dei servizi con i nomi dei file**, struttura delle cartelle e convenzione di naming (una sola lingua). Il `SHARED_CONTRACTS.md` del doc 4 va promosso da rattoppo a fase obbligatoria per tutto il sistema. Ogni feature implementa contro questi contratti, e la compilazione diventa il test d’integrazione gratuito.
3. **Walking skeleton prima delle feature.** Primo agente: costruire l’app minima reale end-to-end (un utente si registra, compra un’azione, il cron la esegue, il saldo si aggiorna — DB vero, zero mock nel runtime). Tutte le feature successive innestano su un sistema che già funziona, e ogni rottura dell’integrazione è visibile subito. Vietare esplicitamente dependency injection “di comodo” con mock hardcoded nel main.
4. **Regression gate meccanico.** Ogni agente, prima di ogni commit, esegue *l’intera* suite (o almeno un sottoinsieme di smoke test end-to-end definito centralmente), non solo i propri test. Idealmente: uno script `npm run verify` fornito dallo skeleton, il cui esito verde è condizione per il commit. L’autocertificazione nel resoconto va sostituita dall’output reale del comando.
5. **Due categorie di test, con infrastruttura fornita.** Separare `test:unit` (mock, sempre eseguibili) da `test:integration` (PostgreSQL effimero via Docker/testcontainers, avviato dallo script stesso). La regola “mai connessioni reali nei test unitari” si applica fornendo il template di setup, non scrivendola in maiuscolo.
6. **Un agente integratore ricorrente.** Ogni *N* feature (non alla fine), un agente con compito esclusivo: eseguire la suite completa, cercare duplicazioni di simboli/servizi, verificare che ogni modulo nuovo sia importato dal runtime (il codice non raggiungibile da `index.ts` è un errore), aggiornare i contratti se necessario. Avrebbe intercettato sia la doppia service layer sia la coda morta con settimane di anticipo.
7. **QA uniforme e tracciato.** Il ciclo gap-analysis va eseguito (e archiviato in `reports/`) per ogni documento, non solo per il primo; e ogni fix post-QA deve ripassare dal regression gate, perché è statisticamente la fase che introduce più regressioni.

8. **Dimensionare le feature sul valore, non sulla decomponibilità.** 42 feature per quest'app significano 41 punti di sutura. Una decomposizione in 10–15 slice *verticali* (es. “ciclo di vita ordine completo, DB→API→UI”) riduce le interfacce interne, che sono esattamente il luogo dove il context rot colpisce.
9. **Il master context deve contenere esempi normativi, non solo regole.** Un file di esempio per servizio, test e rotta (con naming, import, struttura) vincola più di dieci regole in prosa: gli agenti imitano molto meglio di quanto obbediscano.

10 Conclusioni

La pipeline ha dimostrato che una squadra di agenti può produrre, in pochi giorni, la superficie completa di un'applicazione non banale con qualità locale alta: tipi corretti, schema dati fedele, UI completa, storia git esemplare. Ha però anche dimostrato, con evidenza quantitativa, che **la qualità locale non compone automaticamente qualità globale**: 38% dei test falliti, tre implementazioni della stessa logica, un'infrastruttura a code interamente morta e un'integrazione finale che ha rotto ciò che era verde.

Il context rot non ha colpito dove lo si aspettava — dentro il contesto del singolo agente, che anzi la strategia dei contesti piccoli e freschi ha protetto bene — ma **negli spazi tra gli agenti**: nelle interfacce mai concordate, nei requisiti moltiplicati a ogni passaggio documentale, nel “verde” locale mai verificato globalmente, nell'integrazione rimandata all'ultimo momento. La correzione della pipeline non richiede quindi prompt migliori, ma **vincoli meccanici condivisi**: contratti eseguibili definiti prima, uno scheletro funzionante da subito, e un gate di regressione che nessun agente possa autocertificare. Le parole nel contesto si degradano; i contratti compilabili e i test eseguiti no.

A Appendice A — Esito completo della test suite

Esecuzione di `npx jest` sul repository Desktop/prompts (12 luglio 2026):

Test Suites: 81 failed, 1 skipped, 183 passed, 264 of 265 total

Tests: 370 failed, 4 skipped, 589 passed, 963 total

Suite fallite per regressione reale (non ambientale):

- `src/tests/homepage/iteration1.homepage-layout.test.tsx`
- `src/tests/personal-dashboard/iteration1.dashboard-layout.test.tsx`
- `src/tests/personal-dashboard/iteration8.dashboard-composition.test.tsx`
- `src/tests/personal-dashboard/iteration11.page-wiring-actions-and-polling.test.tsx`
- `src/tests/social-interface/iteration5.social-dashboard-composition.test.tsx`
- `src/tests/integration/doc4-remediation.integration.test.ts`

Suite fallite per dipendenza da PostgreSQL reale (868 occorrenze di *Authentication failed against database server*): tutte le suite nelle cartelle sotto `src/tests/` relative alle feature del documento 1 — `friendship-system`, `watchlist-management`, `user-management`, `transaction-processing`, `stock-management`, `portfolio-management`, `permission-management`, `membership-management`, `invitation-system`, `holdings-management`, `historical-data-management`, `group-management` — più `integration/doc1-full-system` e `integration/gap-closure`.

B Appendice B — Inventario delle duplicazioni

- **Servizi portafoglio:** `src/services/PortfolioService.ts` (494 righe, `decimal.js`, errori propri) vs `src/services/portfolio.service.ts` (118 righe, `Prisma.Decimal`) vs logica inline in `src/runtime/runtimeDependencies.ts`.
- **Servizi trading:** `src/services/TradingService.ts` (310 righe) vs `src/services/transazione.service.ts` (183 righe) vs `src/workers/TransactionWorker.ts` (366 righe, mai importato dal runtime).
- **Repository:** `src/repositories/` (11 file, naming italiano: `persona`, `portafoglio`, `gruppo`, ...) vs `src/database/repositories/` (4 file, naming inglese: `UserRepository`, `PortfolioRepository`, ...).
- **Esecuzione ordini:** `TransazioneService.executeOrder` (usata dal runtime) vs `TransactionWorker` (coda `BullMQ`, morta) — duplicazione riconosciuta come item aperto in `TODO.md`.
- **Controlli permessi:** `rbacGroupMiddleware` (costruito, non usato dalle rotte) vs controlli manuali `getEffectiveGroupRole` in `createApiApp.ts` vs `check` in `PermissionService` — tre punti di verità.

C Appendice C — Cronologia essenziale del progetto generato

Ricostruzione dalla storia git (384 commit totali):

Data	Commit	Evento
22-23 feb	14	Feature doc 4 (frontend): profilo utente, sistema immagini; primo scheletro Next.js.
13 mar	18	Ripresa dopo tre settimane: friendship system con guard DB (doc 1).
16 mar	23	Stock, portfolio e transaction management (doc 1).
17 mar	200	Picco: completamento doc 1, gap analysis e due cicli di gap closure; feature doc 2 (coda, worker, servizi); avvio doc 3.
18 mar	128	Feature doc 3 (JWT, RBAC, endpoint, resilienza, rate limiting), integrazione API, remediation <code>TODO.md</code> , wiring runtime reale.
19 mar	1	Ultimo commit (homepage client). Lo stesso giorno inizia lo sviluppo manuale di finApp .

Tabella 6: Timeline del repository generato.